

Kịch bản test vận hành Bridge Node trên tay máy Fanuc CRX thật

Contents

Kịch bản test vận hành trên tay máy Fanuc CRX thật	1
Tổng quan — luồng hoạt động khi mọi thứ chạy đúng	2
0. Lưu ý quan trọng trước khi bắt đầu	3
1. Yêu cầu an toàn — bắt buộc, không tùy chọn	4
2. Chuẩn bị trước khi test	4
2.1 Phần cứng / kết nối	4
2.2 Vật mẫu để test	5
2.3 Phần mềm	6
3. Quy trình test — làm đúng thứ tự, dừng lại nếu có bước không đạt	6
Bước 0 — Xác nhận kết nối robot, chưa chạy phần mềm nhận diện	6
Bước 1 — Test riêng phần nhận diện (camera + yolo_ros), CHƯA launch bridge_node	6
Bước 2 — Launch bridge_node, pipeline TẮT (enable=false)	7
Bước 3 — Bật pipeline, test với 1 vật ở vị trí AN TOÀN, dễ với tới, biết trước	7
Bước 4 — Test dừng an toàn giữa lúc đang di chuyển	8
Bước 5 — Test vật ngoài vùng an toàn	8
Bước 6 — Test với nhiều vị trí/vật khác nhau (chỉ làm khi Bước 3-5 đều đạt)	8
4. Bảng ghi kết quả (điền lại khi test xong)	8
5. Khi có vấn đề	9
6. Kết quả test trên simulator (tham khảo trước khi test thật)	9
Lịch sử thay đổi	10

Kịch bản test vận hành trên tay máy Fanuc CRX thật

Gửi cho: Kỹ sư vận hành tại xưởng **Mục đích:** Chạy thử lần đầu pipeline nhận diện vật (camera + YOLOv8) → tự động di chuyển tay máy Fanuc tới gần vật, trên robot thật.

Tổng quan — luồng hoạt động khi mọi thứ chạy đúng

Đọc phần này trước để hiểu chương trình làm gì tổng thể, trước khi vào các bước test chi tiết ở Mục 3. Đây là mô tả luồng “chạy đúng, không có sự cố” — các bước test chi tiết ở dưới sẽ xác nhận từng phần của luồng này trên robot thật, kể cả các trường hợp bất thường (vật ngoài vùng an toàn, dừng giữa đường...).

Có 3 phần mềm chạy đồng thời, độc lập với nhau:

1. **Camera + nhận diện vật (YOLOv8)** — liên tục chụp ảnh và tìm vật trong khung hình, khoảng 3 lần/giây. Chạy **suốt thời gian**, không dừng lại chờ tay máy, không phụ thuộc tay máy đang làm gì.
2. **Bridge Node** — “cầu nối”, theo dõi kết quả nhận diện, quyết định có nên di chuyển tay máy tới vật đó không, và ra lệnh nếu có.
3. **Tay máy Fanuc (qua `fanuc_driver` + MoveIt2)** — nhận lệnh từ Bridge Node, tự tính đường đi tránh va chạm, di chuyển tới, báo lại kết quả.

Luồng một chu kỳ, khi chạy đúng:

Camera thấy vật hợp lệ (đủ độ tin cậy)

|



Bridge Node tính vị trí 3D của vật trong không gian robot

|



Kiểm tra: vị trí đó có nằm trong vùng an toàn đã cấu hình không?

|

Có ————— Không → BỎ QUA, tay máy không di chuyển, không có gì xảy ra

|



Gửi lệnh di chuyển tới tay máy (qua MoveIt2)

|



Tay máy tự lập kế hoạch đường đi (tránh va chạm) và di chuyển tới gần vật

|

Trong lúc này: Camera **VẪN** đang tiếp tục nhận diện khung hình mới phía sau — nhưng Bridge Node biết tay máy đang bận, nên **KHÔNG**

gửi thêm lệnh mới chồng lên lệnh đang chạy.



Tay máy tới đích → dừng lại → sẵn sàng nhận vật thể tiếp theo

Những gì người vận hành sẽ thấy trong một buổi chạy đúng:

- Không có vật nào trước camera → tay máy đứng yên ở vị trí home, không tự di chuyển.
- Đặt 1 vật (chai nước, cốc...) vào trong vùng camera thấy được và trong vùng an toàn đã cấu hình → sau khoảng 1-2 giây, tay máy tự di chuyển mượt tới gần vật đó rồi dừng lại.
- Nhấc vật ra, đặt vật khác vào chỗ khác (vẫn trong vùng an toàn) → tay máy di chuyển tới vị trí mới.
- Đặt vật ở chỗ ngoài vùng an toàn đã cấu hình → tay máy **không di chuyển** (đây là tính năng an toàn, không phải lỗi).
- Trong lúc tay máy đang di chuyển, nếu ra lệnh tắt hệ thống → tay máy **đi hết đoạn đường đang chạy rồi mới dừng** (không giật/dừng đột ngột giữa đường), sau đó ngồi im tới khi được bật lại.

Toàn bộ luồng này **đã được test và xác nhận đúng trên máy giả lập (simulator)** — xem Mục 6 “Kết quả test trên simulator”. Phần còn lại của tài liệu này (Mục 1-5) là các bước test chi tiết để xác nhận **lại từng phần** của luồng trên khi chạy trên robot thật lần đầu tiên.

0. Lưu ý quan trọng trước khi bắt đầu

Chương trình này **CHỈ** mới được test trên **simulator** (Gazebo/mock hardware trên PC, không có robot thật). Đây là **lần đầu tiên** chạy trên tay máy thật — làm theo đúng thứ tự từng bước dưới đây, **không nhảy cóc**, không tự ý bỏ bước “quan sát an toàn” dù có vẻ chương trình chạy ổn ở bước trước.

3 giá trị sau đang là số đo tạm/giả định lấy từ lúc test trên simulator, CHƯA đo trên robot/camera thật:

- **workspace_bounds** (vùng không gian tay máy được phép di chuyển tới) — đang là hộp $[-1,1] \times [-1,1] \times [0,1.5]$ m. **PHẢI đo lại và thu hẹp** theo tầm với an toàn thật của robot xưởng trước khi chạy Bước 3.

- **table_height** (giả định vật nằm trên 1 mặt phẳng ngang cố định) — đang là 0.0 m. Sai với thực tế xưởng gần như chắc chắn, cần đo chiều cao mặt bàn/băng chuyền thật.
- **grasp_orientation_rpy** (hướng tiếp cận — robot chưa có gripper, chỉ ảnh hưởng orientation khi lập kế hoạch) — đang là $[0,0,0]$. Chưa verify với hướng thật của link flange.

Nếu không đo lại `workspace_bounds` cho hẹp/an toàn thật trước khi chạy, robot có thể tính ra và cố di chuyển tới 1 điểm không như mong đợi.

1. Yêu cầu an toàn — bắt buộc, không tùy chọn

- ☐ Có người vận hành đứng **ngoài tầm với** của robot suốt quá trình test, tay luôn sẵn sàng ở nút **E-Stop**.
 - ☐ Giảm **tốc độ robot xuống mức thấp** (ví dụ override 10-25%) cho toàn bộ buổi test đầu tiên — không chạy tốc độ production.
 - ☐ Dọn sạch vùng làm việc — không có người/vật không liên quan trong tầm với robot.
 - ☐ Test theo **từng bước nhỏ** ở Mục 3 — dừng ngay và báo lại nếu bất kỳ bước nào không đúng kỳ vọng, không tự ý qua bước sau.
 - ☐ Biết chính xác cách nhấn **E-Stop vật lý** trên controller trước khi bắt đầu (không chỉ dùng bằng lệnh phần mềm `/bridge/enable false`).
-

2. Chuẩn bị trước khi test

2.1 Phần cứng / kết nối

- ☐ Robot Fanuc CRX đã xác nhận đúng dòng hỗ trợ, controller có option Stream Motion (J519) + Remote Motion (R912) / S636 External Control Package đã kích hoạt (theo ROADMAP.md Phase 5 — đã xác nhận với xưởng).
- ☐ Kết nối network giữa PC/Jetson chạy ROS2 và Fanuc controller đã thiết lập theo [FANUC ROS 2 Driver Documentation](#) (IP, port, cấu hình phía controller).
- ☐ Camera đã lắp **cố định chắc chắn** (bắt vít/kẹp chặt, không phải để tạm) và **đo và ghi lại** vị trí/góc thật so với gốc robot (`base_link`) — không dùng giá trị tf giả định của bản test simulator. **Lưu ý quan trọng:** toàn bộ hệ thống giả định camera đứng yên tuyệt đối so với robot sau khi đo — nếu camera bị xô dịch (va chạm, rung, tháo lắp lại) sau khi đã đo/cấu hình, mọi tính toán vị trí vật sẽ sai mà **hệ thống không tự phát**

hiện được (không có cơ chế tự kiểm tra camera có bị lệch hay không). Nếu nghi ngờ camera bị xô dịch, phải đo lại từ đầu trước khi tin tưởng kết quả.

- Đo chiều cao mặt phẳng nơi vật sẽ được đặt (bàn/băng chuyền) so với `base_link` → cập nhật `table_height`.
- Đo/ước lượng vùng không gian **an toàn và chắc chắn robot với tới được** → cập nhật `workspace_bounds` **hẹp hơn** tầm với thật (có biên độ dự phòng), không dùng nguyên hộp mặc định.

2.2 Vật mẫu để test

Model nhận diện hiện tại (YOLOv8 pretrained COCO) chỉ nhận diện được **80 loại vật đã được train sẵn** — [danh sách đầy đủ](#). Không phải class nào cũng dễ nhận diện như nhau.

- **Chọn vật cụ thể, dễ nhận diện** — ưu tiên vật hay gặp, hình dạng rõ ràng, không trong suốt/phản chiếu mạnh: `bottle` (chai nước), `cup` (cốc), `cell phone`, `book`, `keyboard`, `mouse`, `scissors`. Tránh vật nhỏ/mảnh (ví dụ `fork`, `spoon`) hoặc vật dễ lẫn với nền ở lần test đầu.
- Chuẩn bị **2-3 vật khác class nhau** (ví dụ: 1 chai nước + 1 cuốn sách + 1 cái cốc) — **không đặt 2 vật cùng class cùng lúc trong khung hình** (ví dụ 2 chai nước) ở các Bước 3-6: hệ thống hiện tại không phân biệt được 2 vật cùng loại đứng cạnh nhau (xem `DDB-bridge-node.md` Open Issue #7), sẽ gây nhầm lẫn vị trí, không phải lỗi nhưng chưa xử lý được tình huống này.
- **KHÔNG dùng vật đặc thù của xưởng** ở lần test này — model chưa được train cho vật đó, sẽ không nhận diện được hoặc nhận diện nhầm sang class gần giống. Việc train riêng cho vật của xưởng (fine-tune hoặc YOLO-World) là công việc khác, làm sau khi pipeline cơ bản đã chạy ổn.

Vị trí đặt vật (quan trọng, ảnh hưởng trực tiếp tới việc nhận diện có ra kết quả đúng không):

- Đặt vật **trong khung hình camera nhìn thấy rõ** (không bị che, không ở rìa khung hình) — kiểm tra bằng cách xem ảnh trực tiếp (xem Bước 1).
- Khoảng cách vừa phải: **không quá gần** (vật chiếm gần hết khung hình / bị cắt mép — bbox sai) và **không quá xa** (vật quá nhỏ trong khung hình — model dễ bỏ sót hoặc score thấp). Bắt đầu ở khoảng cách mà vật chiếm khoảng 1/6 – 1/3 chiều rộng khung hình.
- Nền phía sau vật **không quá lộn xộn**, đủ ánh sáng đều (tránh ngược sáng — vật tối om trước cửa sổ/đèn sáng phía sau), tránh bóng đổ mạnh che vật.

- ❑ Vật nằm trong vùng mà sau khi tính tọa độ 3D sẽ rơi vào `workspace_bounds` đã cấu hình (Mục 2.1) — nếu không chắc, cứ test trước, xem log cảnh báo BR004 để biết pose tính ra là bao nhiêu, rồi điều chỉnh vị trí vật hoặc `workspace_bounds`.

2.3 Phần mềm

- ❑ `fanuc_driver` build cho robot thật, launch **KHÔNG dùng** `use_mock:=true` (cờ đó chỉ dùng cho giả lập trên PC).
 - ❑ Cập nhật file `params` của `bridge_node` (`workspace_bounds`, `table_height`, `camera_frame`, `grasp_orientation_rpy`) theo số đo thật ở Mục 2.1 — **không dùng file mẫu của bản test simulator**.
 - ❑ `enable_on_start` đặt về `false` trong lần chạy đầu (pipeline khởi động ở trạng thái TẮT, chỉ bật thủ công khi đã sẵn sàng quan sát — xem Bước 3).
-

3. Quy trình test — làm đúng thứ tự, dừng lại nếu có bước không đạt

Bước 0 — Xác nhận kết nối robot, chưa chạy phần mềm nhận diện

1. Launch `fanuc_driver` (không mock) — xác nhận controller nhận kết nối, không có lỗi.
2. Dùng công cụ có sẵn (RViz Jog hoặc teach pendant) di chuyển robot một khoảng nhỏ, xác nhận điều khiển được và an toàn.
3. **Đạt:** robot di chuyển đúng lệnh, dừng đúng lúc. **Không đạt:** dừng lại, không qua bước 1, báo lại vấn đề kết nối/driver.

Bước 1 — Test riêng phần nhận diện (camera + yolo_ros), CHƯA launch `bridge_node`

1. Launch camera driver thật + `yolo_bringup yolov8.launch.py`.
2. **Xem ảnh trực tiếp có vẽ khung nhận diện** (không chỉ xem số liệu) — dùng `rqt_image_view`, chọn topic `/yolo/debug_image`. Đây là cách nhanh nhất để biết camera có thấy đúng vật, khung hình có che/lệch không, trước khi quan tâm tới tọa độ 3D.
3. Đặt 1 vật mẫu (theo Mục 2.2) trước camera, theo đúng khoảng cách/vị trí đã hướng dẫn.
4. Xem topic `/yolo/detections` (hoặc nhìn khung hình trong `/yolo/debug_image`) — xác nhận có phát hiện đúng vật, đúng tên class, score hợp lý (>0.5), khung bao (bbox) ôm sát vật chứ không lệch sang vật khác.

- Thử di chuyển vật qua 2-3 vị trí khác nhau trong vùng dự kiến sẽ test — xác nhận detect ổn định ở mọi vị trí, không chỉ 1 chỗ may mắn.
- Đạt:** detect đúng vật, đúng khung, ổn định qua vài giây, ở nhiều vị trí thử. **Không đạt:** dừng lại, xử lý theo bảng dưới trước khi qua Bước 2.

Xử lý khi không đạt:

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Không có detection nào	Vật ngoài khung hình / quá xa / quá tối	Kiểm tra <code>/yolo/debug_image</code> xem có thấy vật không; tăng sáng; đưa vật gần lại
Score thấp (< 0.5) liên tục	Vật quá nhỏ trong khung hình, góc chụp lạ, ảnh mờ/rung	Đưa vật lại gần hơn; giữ camera/vật ổn định; lau ống kính
Detect sai class (ví dụ nhận nhầm chai thành cốc)	Vật che khuất 1 phần, góc chụp không điển hình, ánh sáng làm biến dạng hình	Đổi góc đặt vật; thử vật khác trong danh sách Mục 2.2
Detect nhảy liên tục giữa “có” và “không”	Ở ranh giới threshold, ánh sáng không ổn định (đèn nhấp nháy, ánh sáng tự nhiên thay đổi)	Cải thiện ánh sáng ổn định hơn; có thể cần chỉnh <code>score_threshold</code> (báo lại người viết chương trình trước khi tự đổi)

Bước 2 — Launch bridge_node, pipeline TẮT (enable=false)

- Launch `bridge_node` với params đã cập nhật số đo thật (Mục 2.1, 2.3).
- Xác nhận node lên không lỗi, topic `/bridge/status = idle`.
- Không đặt vật vào camera ở bước này** — chỉ xác nhận node chạy ổn định, không tự gửi lệnh gì.

Bước 3 — Bật pipeline, test với 1 vật ở vị trí AN TOÀN, dễ với tới, biết trước

- Người vận hành đứng ngoài tầm robot, tay ở gần E-Stop.
- Đặt 1 vật mẫu ở vị trí **gần giữa workspace đã cấu hình**, dễ quan sát, chắc chắn an toàn.
- Gọi `ros2 service call /bridge/enable std_srvs/srv/SetBool "{data: true}"`.

4. Quan sát: robot có di chuyển đúng hướng vật, tốc độ chậm (theo override đã giảm ở Mục 1), dừng lại gần vật không giạt cục.
5. **Đạt:** robot dừng đúng gần vị trí vật (sai lệch chấp nhận được, ghi lại số đo thật).
Không đạt: nhấn E-Stop ngay, ghi lại sai lệch/hướng đi sai, báo lại — chưa qua bước 4.

Bước 4 — Test dừng an toàn giữa lúc đang di chuyển

1. Lặp lại Bước 3, nhưng lần này khi robot **đang di chuyển** (chưa tới đích), gọi /bridge/enable false.
2. **Kỳ vọng** (đã verify đúng trên simulator, giờ xác nhận trên robot thật): robot **hoàn thành nốt** chuyển động đang chạy (không dừng đột ngột giữa đường), sau đó không nhận lệnh mới cho tới khi enable lại.
3. **Đạt:** đúng như kỳ vọng. **Không đạt:** đây là vấn đề an toàn quan trọng — dừng test, báo lại ngay.

Bước 5 — Test vật ngoài vùng an toàn

1. Đặt vật mẫu ở vị trí **biết trước là ngoài** workspace_bounds đã cấu hình.
2. Bật pipeline, quan sát.
3. **Đạt:** robot **không di chuyển**, chỉ log cảnh báo. **Không đạt:** vấn đề an toàn nghiêm trọng — nhấn E-Stop ngay, báo lại.

Bước 6 — Test với nhiều vị trí/vật khác nhau (chỉ làm khi Bước 3-5 đều đạt)

1. Lần lượt thử 2-3 vật mẫu khác nhau (Mục 2.2), mỗi lần 1 vị trí khác nhau trong workspace an toàn.
2. Ghi lại: vật, vị trí đặt, robot có tới đúng không, sai lệch ước lượng.

4. Bảng ghi kết quả (điền lại khi test xong)

Bước	Đạt/Không đạt	Quan sát thực tế	Người test	Ngày
0 — Kết nối robot				
1 — Nhận diện riêng				

Bước	Đạt/Không đạt	Quan sát thực tế	Người test	Ngày
2 — Bridge tắt				
3 — Di chuyển tới vật				
4 — Dừng an toàn giữa đường				
5 — Vật ngoài vùng an toàn				
6 — Nhiều vật/vị trí				

5. Khi có vấn đề

- **Bất kỳ hành vi bất thường/nguy hiểm** → nhấn E-Stop vật lý ngay, không chờ báo cáo trước.
- Báo lại: bước nào, log của `bridge_node` (`ros2 topic echo /bridge/status`, log terminal), ảnh/video nếu có, giá trị `workspace_bounds/table_height` đang dùng.
- Không tự sửa `workspace_bounds/table_height/grasp_orientation_rpy` để “cho chạy được” mà không báo lại — các giá trị này ảnh hưởng an toàn, cần xem lại cùng người viết chương trình trước khi đổi.

6. Kết quả test trên simulator (tham khảo trước khi test thật)

Toàn bộ luồng ở phần “Tổng quan” đã chạy và xác nhận đúng trên simulator (Gazebo mock hardware, không phải robot thật) trước khi gửi kịch bản này. Xem kèm trong gói giao (deliveries/):

- `QA-testcases.csv` — 21 test case, tất cả Pass (đơn vị + tích hợp trên simulator)
- Ảnh chụp/video màn hình simulator lúc chạy thật (RViz + log)
- `docs/design/DDB-bridge-node.md` — thiết kế chi tiết, bao gồm 2 lỗi thật đã phát hiện và fix trong lúc test (BR007: action server chết làm treo vô hạn; BR008: gửi lệnh chông khi tay máy đang di chuyển)

Lưu ý quan trọng: test trên simulator dùng vị trí/camera/mặt phẳng **giả định**, không phải số đo của xưởng — đây là lý do Mục 2.1 yêu cầu đo lại trước khi chạy thật.

Lịch sử thay đổi

Ngày	Phiên bản	Thay đổi	Người thực hiện
2026-09-11	1.0	Khởi tạo — kịch bản test lần đầu trên robot thật, sau khi xưởng xác nhận robot đúng dòng CRX + có option phần mềm cần thiết	Hoang Duc
2026-09-12	1.1	Thêm phần “Tổng quan — luồng hoạt động khi mọi thứ chạy đúng” lên đầu tài liệu; thêm Mục 6 tham chiếu kết quả test simulator trong gói giao	Hoang Duc
2026-09-12	1.2	Đổi tên Mục 0 cho rõ nghĩa hơn (“Lưu ý quan trọng trước khi bắt đầu”); sửa lỗi PDF (bảng bị xoay ngang trang không cần thiết ở Mục 4/Xử lý khi không đạt/Lịch sử thay đổi — ép giữ portrait)	Hoang Duc